

FACULTY OF ENGINEERING AND GREEN TECHNOLOGY

UGEA2633 MICROPROCESSOR & MICROCONTROLLER SYSTEM / EMBEDDED SYSTEM

ASSIGNMENT REPORT

TOPIC 1: ELEVATOR CONTROLS

Prepared By:

Student Name	Student ID
Ng Yu Meng	[redacted]
Boon Hsien Long	[redacted]

Lecturer: Dr Lee Han Kee, Dr Lee Yu Jin

Date: 8/8/2024

Table of Contents

Introduction....3
Hardware Design....4
Schematic Diagram....6
Combined Circuit....7
Software Design....8
Additional Feature Implementation....10
Flowchart....11
Conclusion....15
Appendices....16
Results....16
Assembly Program....18
References....20
Assignment Demo Link....20

Introduction

This assignment focuses on designing and implementing a multi0floor elevator control system using a 8051 microcontroller. Intel invented the 8051 microcontrollers in 1981. A type of integrated circuit is the microcontroller, which has 40 pins, a dual inline package (DIP), 128 bytes of RAM, 4KB of ROM, and two 16-bit timers. It has addressable and programmable 4-parallel 8-bit ports, as required. (Agarwal, 2021) Elevators are important for any city buildings as well as any high-rise buildings, where convenience and safety always come first. The system designed in this assignment is to able to control an elevator that supports minimum three floors, adding hardware components that are necessary for practical use. This includes input and output systems for the elevator's internal and external control.

External of the elevator, each floor except the highest and lowest floor has "Up" and "Down" buttons to call the elevator. These buttons allow the users to request travel upwards or downwards. Moreover, the external of the elevator at all floors will show the elevator is currently moving upwards or downwards, and which floor the elevator is currently at. This is to ensures that users are informed about the elevator's position and movement before entering.

Internally, the elevator has a panel for floor selection, which allow the users to select their floor that they wished to go. Other than that, internal of the elevator will provide continuous feedback about the elevator's current position and direction that the elevator is currently travelling. In case of any accident happened to anyone that are travelling to other floor using the elevator, the system included an emergency button that disables all operation. On the other hand, there will be a reactivation button to resume normal elevator operations once the emergency is settled. The system is also designed with power saving mode, as the elevator lights (represented by LEDs) automatically switch off after nobody using the elevator for a period but reactivate whenever the up or down button is pressed.

Other than just showing LEDs, the DC motor plays a dual role which serve as the physical movement mechanism and as an up or down indicator. The overall goal of this assignment was to create a practical and effective elevator system that puts users' safety and experience first using hardware interfacing and microcontroller programming.

In short, our design aims to create an elevator system that resembles the real life application of one by using multiple required systems.

Hardware Design

This assignment is divided into two main sections: the hardware connection section and the programming software section. Several outputs for the hardware interface, including the emergency light, elevator light and up/down indicator, are displayed in LEDs. In this assignment, the floor indicator is displayed using the Seven Segment method and the buzzer will also be added whenever the emergency mode triggered. Other than that, to show more about the elevator is going up or down, we also added a DC motor into it by showing which direction of the elevator is going by telling which direction of the DC motor is spinning. In the meantime, all inputs we are using will be only push buttons. Example such as, internal floor buttons, internal emergency and reactivate buttons, external up or down button and more.

To control the DC motor direction of our elevator system, the DC motor must connect to one of the L293D IC to determine its turning direction. Since the L293D is a dual H-bridge motor driver, you may adjust the motor's rotation direction by changing the direction in which current flows through it. Therefore, following our programming code it shows that the DC motor will turn in a clockwise direction whenever the elevator is travelling upwards and anti-clockwise when the elevator is travelling downwards. Besides L293D is being used, each of the LED are connected to a NPN transistor. Three-layered NPN transistors are a kind of bipolar transistor used in signal amplification. (*What is NPN Transistor? Definition, types & applications.*, 2021b) It is a device that is controlled by current control. NPN is a code name for negative-positive-negative transistors. In this structure, a p-type semiconductor is fused between two n-type semiconductor materials.

Briefly talk about the circuit, the circuit divided into parts like external and internal of the elevator. This elevator system is to build as realistic as possible like the real-life elevator. When the elevator at floor one, the external of the elevator will only have “UP” button which means that the elevator currently in floor 1 can only be travelling upwards whenever the elevator reached floor 1. Moving on to floor 2, the external of the elevator will have two buttons which are “UP” and “DOWN”. Users can choose between these two buttons to travel upwards to floor 3 or downwards back to floor 1 whenever the elevator reached floor 2. Finally, to floor 3, same concept as floor 1, whenever we are at the top floor of the building, we can only be travelling downwards. Therefore, the external of the elevator at floor 3 will only be having “DOWN” button to travel downwards whenever the elevator reached floor 3. As we know, microcontroller 8051 has four parallel 8-bit ports which are addressable and programmable. There will be 3 parallel 8-bit ports being used, two of the ports, P1 and P2 is

to show the ideal output of our elevator systems, meanwhile P3 will be the input ports. P1.2 is an output pin which connected to two LEDs and will be high whenever the is elevator moving downwards. Before connecting pin P1.2 to all the LEDs, P1.2 will first connect to one of the NPN transistors' base, meanwhile the collector of the transistor is connected to 5V and finally the current will flow through the emitter of the transistor towards pin 2 of the L293D IC. Whereas the two LEDs are connected to the same pin as pin 2 then connected to the ground. In comparison with P1.2, P1.3 indicates "go up," meaning that the elevator is now moving upward whenever this port is high. Same as P1.2, P1.3 will also connect to the transistor the same way as P1.2 to the pin 7 of L293D IC. And the two LEDs will also share the same pin as L293D then connected to the ground. Moving on to the next port, P1.4 attached to a buzzer will start beeping whenever the port has set high, the buzzer will not stop buzzing until the internal reactivate push button on the bread board been hit. At the same time while the buzzer is buzzing, P1.5 attached to a LED turns on as well to show the elevator is in emergency mode. The P1.6, the final output port for P1, is connected to an LED that will always be on when the elevator is in use. However, the elevator will automatically enter our "Energy Saving" mode when the LED turns off after a short time of inactivity. Moving on to port P2, we are using different code every time the floor changes instead of utilizing any bits to display different results. P2 is connected to 2 seven segments which is for externally and internally. Whereas the seven segments will only be showing three numbers and one alphabet in this elevator system. Whenever the system reached floor 1, the seven segments will be showing "1", seven segment shows "2" when elevator system reaches floor 2 and finally the seven segments will show "3" when the elevator system reaches floor 3. On the other side, the seven segments will show letter "E" when the internal emergency push button is pushed. This is the end of the output port. We will only use P3 for all input ports, and different bits for separate input. A push button that reactivates the signal in the event of an emergency is linked to P3.2. P3.2 will only be hit in the event of an emergency. Meanwhile, P3.3, an external interrupt linked to a push button, will stop all functions and switch to emergency mode right away. For the last three ports which connected to three different push buttons, will be shown inside the elevator. Where P3.5 is to floor 1, P3.6 is to floor 2, P3.7 is to floor 3.

After discussed about elevator up/down system, our group has successfully combined with another group with the topic of elevator door close/open system which combined into a fully workable realistic elevator system. First and foremost, system from topic 1, up/down system, will press any buttons from the breadboard, then move to the ideal floor. After reaching to the ideal floor, topic 2, door open/close system, will receive a signal to

acknowledge that they will need to open the door to let the elevator passenger to enter or exit the elevator. Moving on, after topic 2 has sense that no more passenger is going to leave nor enter the elevator, the door will start closing. All closing and opening doors output are shown in the DC motors and LEDs that has patched on the breadboard. After the door close, the elevator will start moving the elevator to the ideal floor that the passenger wants to go. When emergency happens, where passenger pressed the emergency button, the door will not open or close no matter how many times the passenger inside the elevator pressed the open or close door button. This is to ensure the passengers' safety, to protect the passenger to fell of from the elevator.

Schematic Diagram

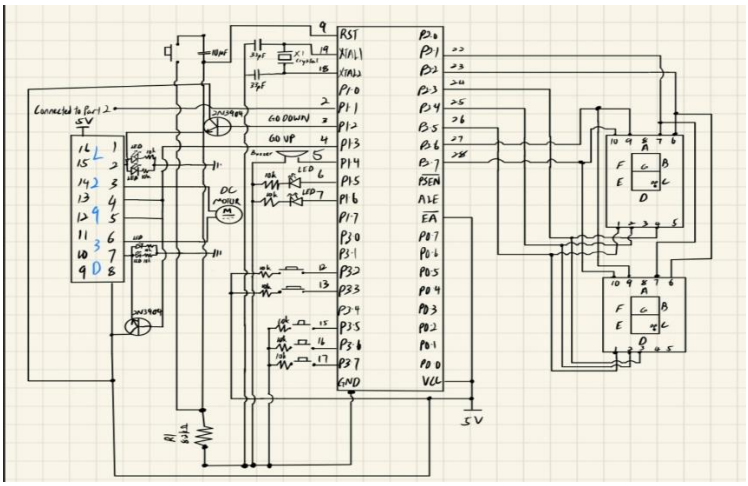


Figure 1.7: Schematic diagram of Elevator up/down system

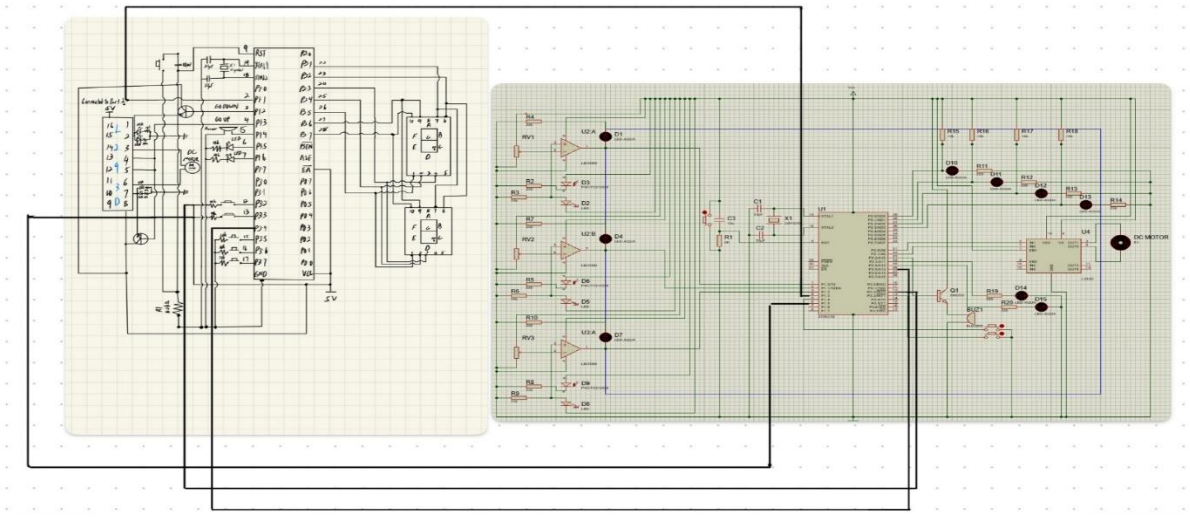
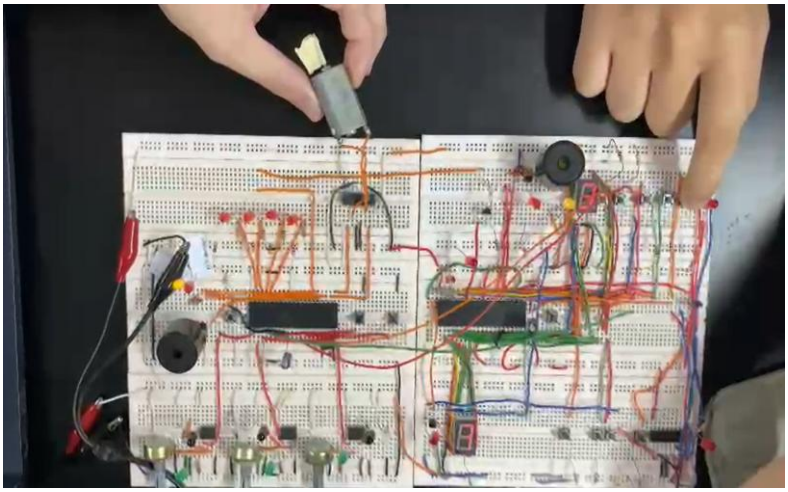
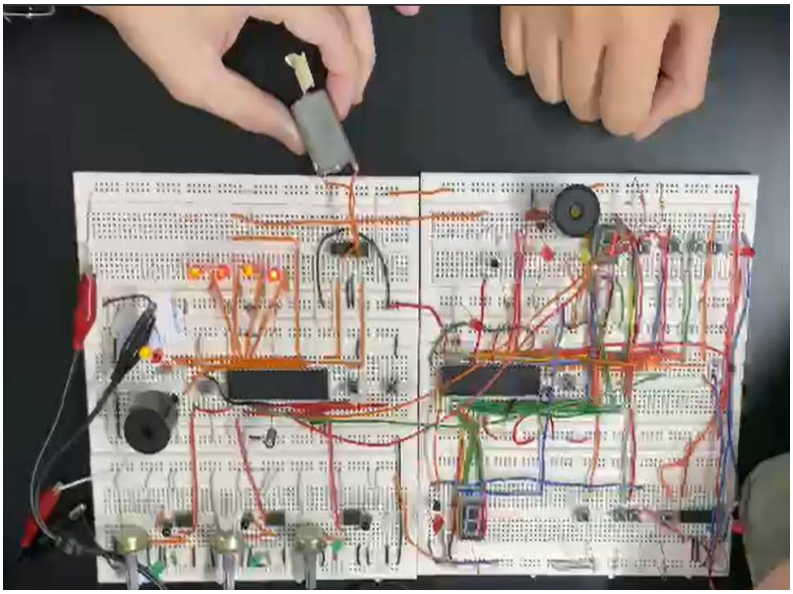


Figure 1.8: The combined schematic diagram of part 1 and part 2

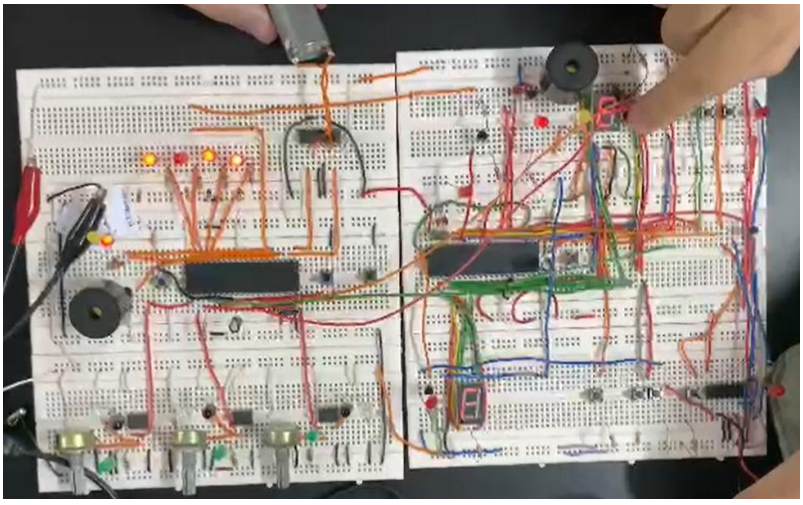
Combined Circuit



The elevator when it is at third floor



The elevator when it is at first floor with 16 passengers



The elevator in Emergency State

Software Design

The program starts by initializing 4 ORGs. 00H which indicates where the program should start without interrupt, 003H receiving external interrupt 1 from pin 3.2 when 0013H receives external interrupt 1 from pin3.3. The 003H indicates where the program should return to under circumstances. It can be further separated into 11 modules namely SETUP, LOOP, FlOOR1 2 and 3, DELAY, OPEN, REPEAT, EMERGENCY, RESUME and SLEEP. SETUP is responsible for setting up all parameters and variables used in the program when LOOP enters prepares the microcontroller for external input. Once the external input is received it will execute the respective actions in FLOOR1 FLOOR2 or FLOOR3 module. The DELAY subroutine is a module that would be called every time a process requires a pause. OPEN function sends a signal to Part II of our assignment which in charge of controlling the door. REPEAT module serves the function of fixing the looping required. EMERGENCY stops the 8051 from running completely when RESUME reactivates the function, SLEEP is called when there is no input of any kind after a certain period. Several Registers has been used. R0 is in charge of storing the value of current floor in respect to the seven-segment display, when R2 is used to store current floor value. R3-R5 is used for DELAY when R7 is to store the difference between each floor. R1 and R6 is not utilized.

The SETUP module sets P1 as output pin by turning all pin to LOW by setting it to #00H. A current sink has been used for P3 by setting it to be 0FFH. This will allow an input to be triggered whenever there is a pulse of 1 given to 8051 microcontrollers. IE and IP are set to 10000111B and 00000001B respectively. The purpose of such action is to enable the interrupt using IE and prioritize P3.2 (External Interrupt 0) over P3.3(External Interrupt 1) Initially we assume the state of the elevator to be at 3rd floor whenever the elevator has been implemented for the first time. In order to clearly display its current floor level, a seven segment is used. The P2 is connected directly to the seven segment and hence it was set to 09E (10011110) displaying 3 on the seven-segment display. And R0 stores a value of 3

The LOOP module initialize itself by setting P1.6 as HIGH which turn on the idle LED. Instead of using timer interrupt, a delay, which is a type of polling programming, has been implemented. However, unlike ordinary nested delay, a condition has been placed between each delay. When an input from either P3.5, P3.6, P3.7 has been implemented. It will proceed to the subroutine FLOOR1 FLOOR2 FLOOR3 respectively. The inputs will be determined via JNB which jump to the respective subroutine if no voltage has been detected. As our design uses current sink, a input will be detected when the push button that connect to the voltage supply is pressed. In the unfortunate incident that when more than 1 input has

been implemented at the same time, the priority will be at Floor 1 then to Floor 2 and lastly Floor 3. After no external inputs has been detected for 16 seconds, it will proceed to SLEEP module.

EMERGENCY subroutine is a special module which uses the External Interrupt 1. Whenever the External Interrupt 1 is triggered via a signal given to P3.3, The External Interrupt 1 will be turned on and the program will proceed to ORG 0013H. As the module related to 0013H is the EMERGENCY subroutine, it will perform a long jump which ignores everything in the middle and straight away execute the EMERGENCY subroutine. An alphabet E will be displayed on the seven segment display via 0F2H(10011110) is sent from P0. P1.6 which controls the internal LED will also be shut down along with P1.3 and P1.2 which is the motor control. The buzzer will start to beep along with the red LED connected to P1.4 and P1.5 respectively. It will turn itself on and off following a frequency via the delay control. It will only exit the EMERGENCY state when P3.2 is pressed and enters the RESUME state. P3.2 is the external interrupt 0 which clear the buzzer and the LED and display the current floor value by directing the using the value stored in R2.

The SLEEP module will turn off everything except the current floor. It will then enter an infinite loop which the 8051 microcontroller stays idle until an external input from P3.5, P3.6 or P3.7 has been triggered. It will then return to the main LOOP function which await further orders. The OPEN subroutine is a successful attempt to communicate with Part II of the assignment which is in charge of the closure and opening of the door. A signal will be sent to their microcontroller from our P1.1 when it will then enter a state of pause which await feedback from Part II via P3.4 of 8051 of Part I, symbolizing that the door is closed and is ready to move. P1.1 will then be cleared to 0 to symbolize the door closing and start to move. The REPEAT subroutine will repeat the delay loop based on the difference between current floor and the floor calling for the elevator.

The FLOOR1 and FLOOR3 shares the completely same logic. However, FLOOR3 can only go down hence the motor control will always be P1.3 when FLOOR1 can only go up hence the motor control will only be P1.2. Whenever the subroutine is called, it will first call the OPEN subroutine to open the door. After a signal indicating that all passenger is inside the elevator is sent from Part II, the microcontroller will subtract the current floor value with the floor value that the passenger is trying to go. For FLOOR1, it will subtract the value of the current floor with 1 when for FLOOR 3 it will use 3 to subtract the value of current floor stored in R2 The value which is stored in R7 will then be used to determine how many cycles

it should run. The 8051 will then send a signal to the L293D via P1.2(For FLOOR1, indicating going up) or P1.3(For FLOOR3, indicating going down) it then will wait for a certain period depending on the REPEAT module. After the REPEAT ended, it will turn off the motor indicating that the floor has been reached. Another OPEN subroutine will be called again to open the door and allow the passenger to exit. The current floor value, 1 for FLOOR1 and 3 for FLOOR3 will be stored. When a 1 will be displayed on the seven-segment via an 00C (00001100) for FLOOR1 and 9E(10011110). After everything has been done, it will proceed back to LOOP module. If the current floor is equivalent to the floor where the elevator is called, the elevator will open the door then closed again, no further actions will be taken. This process is being done via the JZ function.

FLOOR2 has a similar concept as FLOOR 1 and FLOOR3. However, the subtraction will have three results, larger, smaller, or equal. This is determined by JC and JZ. JC refers to Jump if Carry. When the results of subtraction between current floor and floor that is being called is lesser than 0 it means the caller is at a higher altitude compared to the current floor level and hence the elevator needs to move up, triggering the P1.3 going up. If the difference between two floors is 0 that means that passenger and the elevator are at the same floor. Hence the door will only open. This is done by the JZ function which Jumps if Zero. If the results are neither larger or equal, it means that the person calling for the elevator is at first floor, hence the 8051 will send out a signal to L293D via P1.3, controlling the elevator to proceed downwards. After it reached, 2 will be displayed on the seven segments via 0B6H (10110110). A value of 2 will be stored in R0 for implementation in the future process.

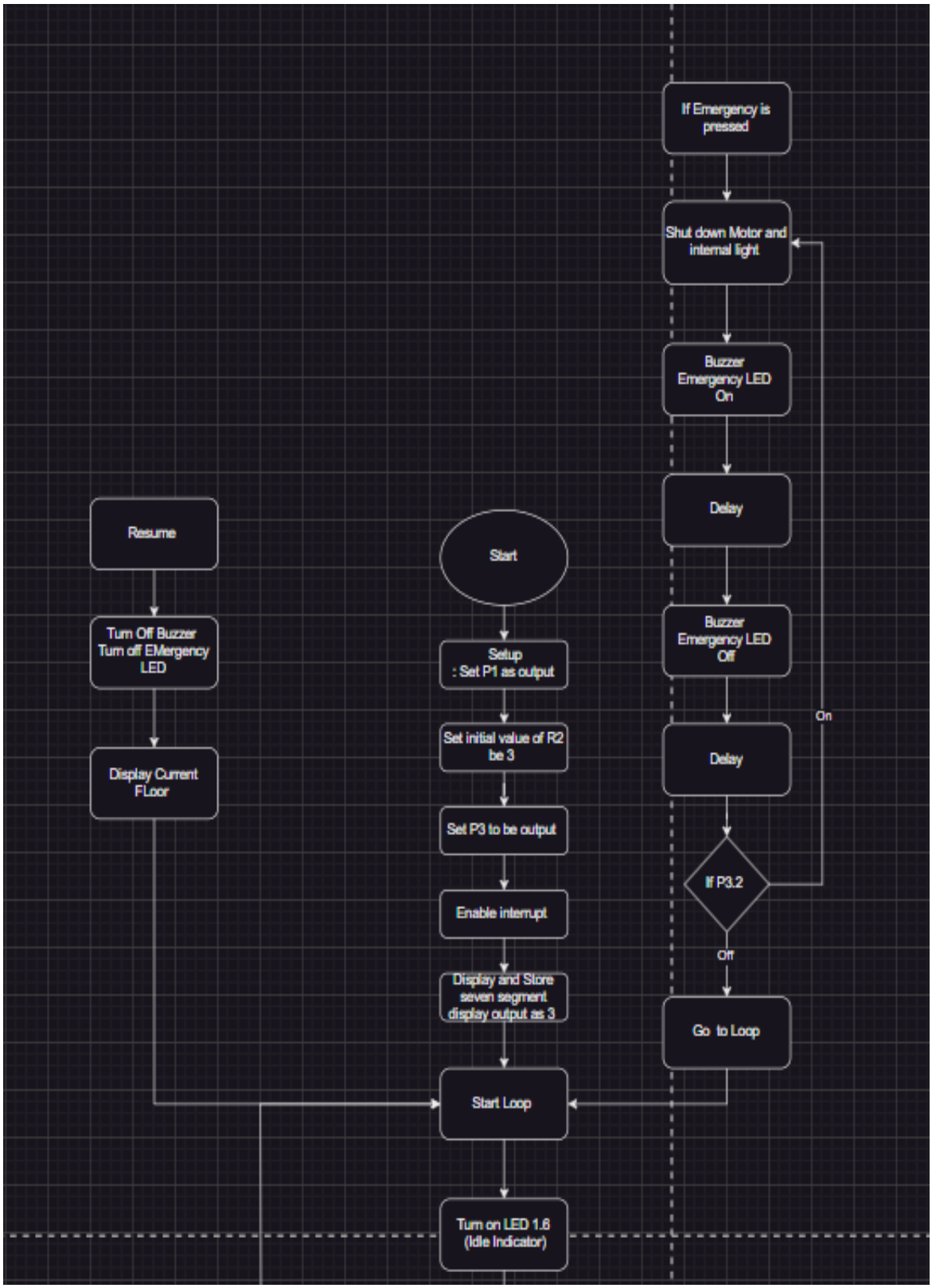
Additional Feature Implementation

Two additional features have been implemented in our design. Namely the usage of a DC motor via the connection of H-Bridge using the L293D motor driver and a buzzer

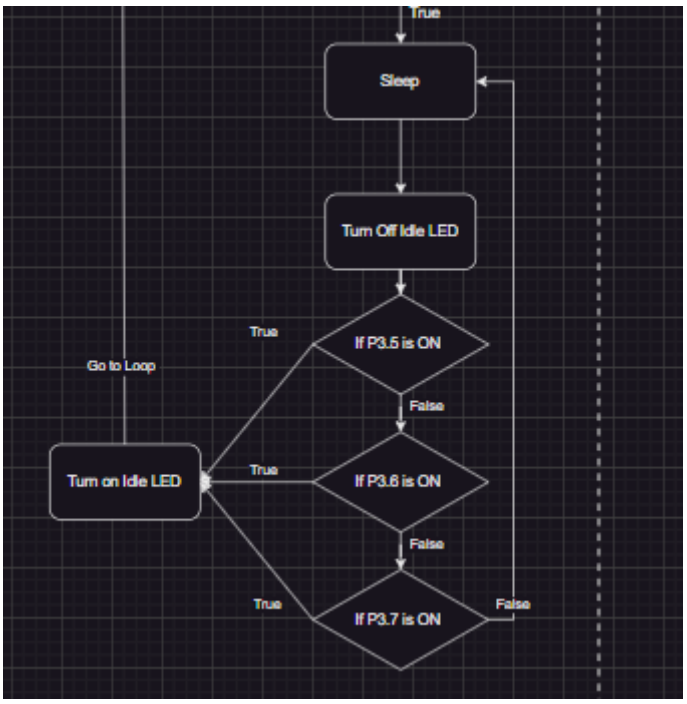
The purpose of implementing the motor is to imitate the actual representation of an elevator. In the real-life application, the elevator uses a motor to carry the carriage, being the object that carries the passenger, up and down between different floors. In order to simulate such actions, the determining factor is the direction of motor. This is done via the connection of H-bridge. H-bridge consists of 4 switches connected to the same motor The H bridge allows the switching of polarity and hence it can be used to control the direction of the motor by applying current via different polarity locations. (David, 2023)

The purpose of the buzzer that beeps to a certain frequency is to create an intense feeling to the external individuals to indicate the emergency status within the elevator itself.

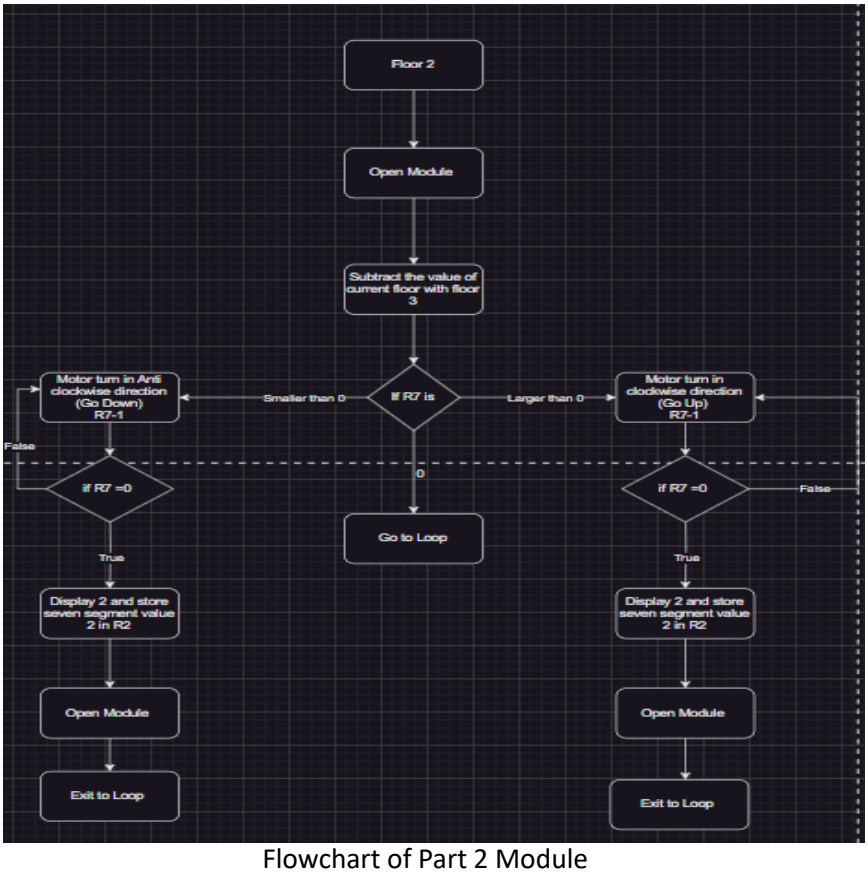
Flowchart



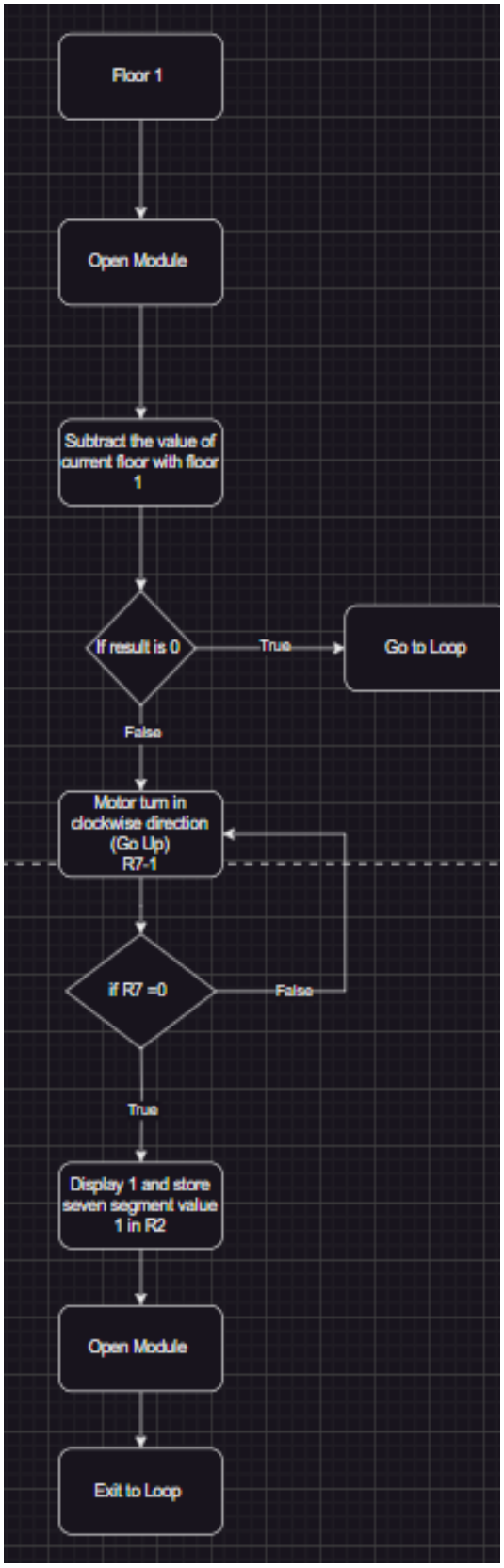
Flowchart of Start Module



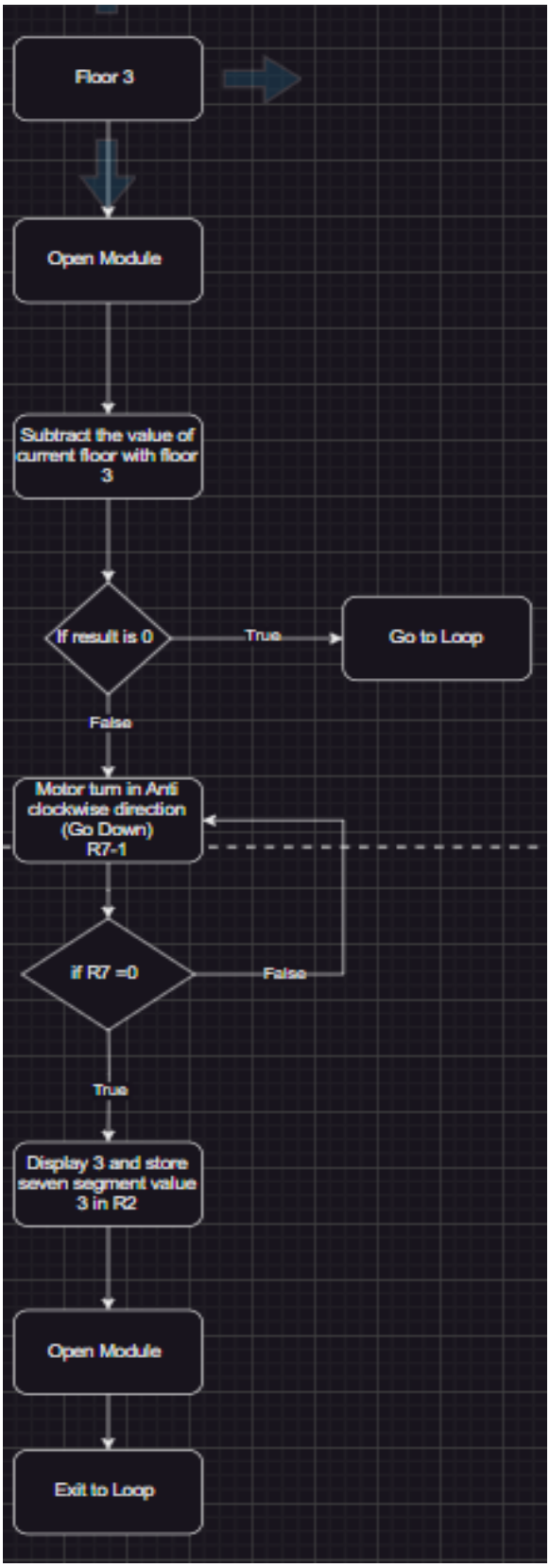
Flowchart of Idle Module



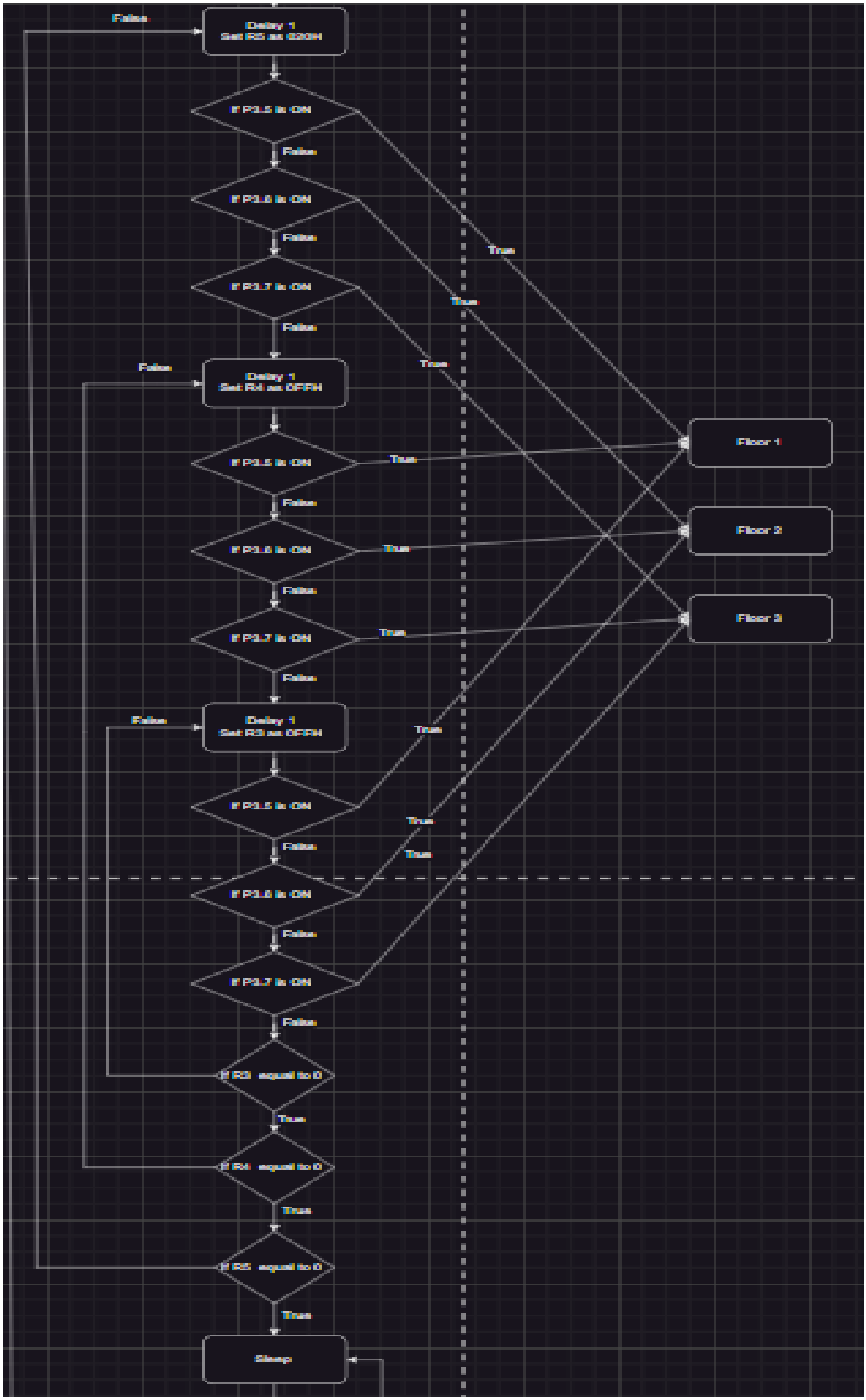
Flowchart of Part 2 Module



Flowchart of Part 1 Module



Flow chart of Part 3 Module



Flowchart of the Main Module

Conclusion

To sum up, this elevator system project showcases a comprehensive hardware integration that makes use of the 8051 microprocessors. With abilities including floor indication, direction control, emergency management, and energy-saving modes, the system successfully simulates a real elevator. The direction of the DC motor, which determines whether the elevator moves up or down depending on the flow of current, is mostly controlled by the L293D IC. In order to maintain appropriate current regulation and enable effective signal amplification while protecting the microcontroller, NPN transistors were used for the LEDs.

All the system's input control is done using push buttons, which makes it easier for users to interact with internal and external buttons for direction selection, floor selection, and emergency handling. The system is user-friendly since it uses LEDs, seven-segment display, and a buzzer for emergencies to visually communicate outputs. Transistors drive LEDs that show the elevator's motion and status, and a seven-segment display makes it obvious which floor it is on or whether an emergency is present.

The system performs smooth input-output connection using three different 8051 ports (P1, P2, and P3). The outputs for the motor control, emergency light, and buzzer are handled by port P1, and the seven-segment displays are handled by port P2. Port P3, which reads signals from different push buttons and controls the system's key functions including floor selection and emergency mode, is in charge of managing the input.

In order to create an operational and realistic elevator system, this project effectively hits a balance among programming logic and hardware components. It also points out important ideas of microcontroller-based control systems, like motor control, signal amplification, and emergency management, while placing a major focus on user interaction and safety features.

Appendices

Results

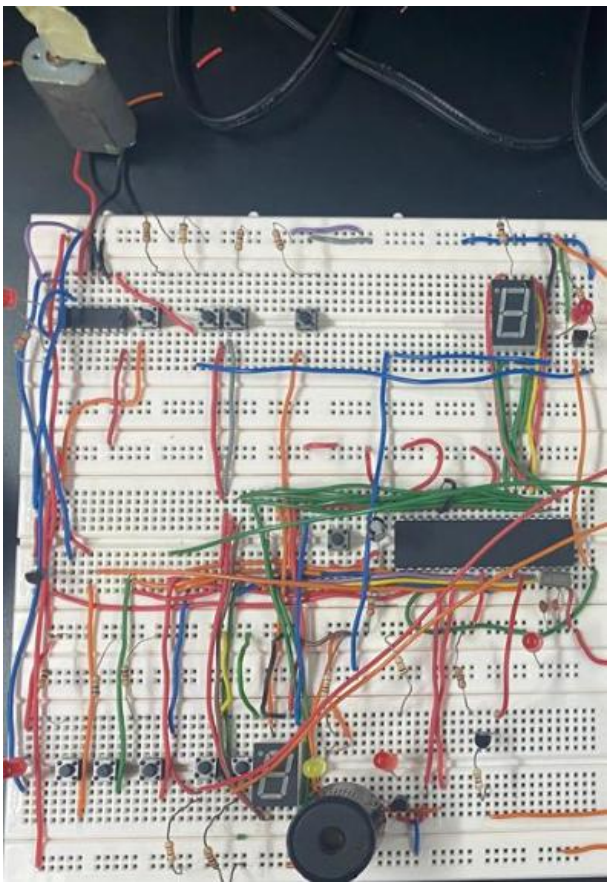


Figure 1.1: Final circuit of up/down elevator system

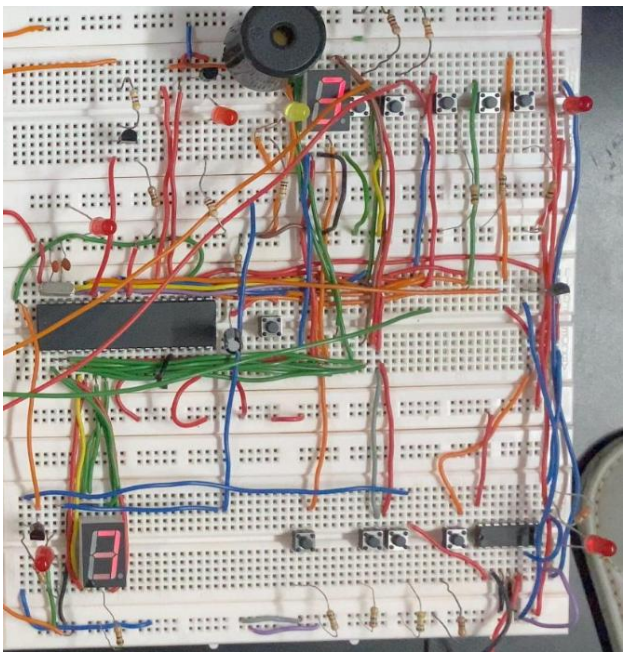


Figure 1.2: Seven Segment shown “3” when elevator system currently at floor 3

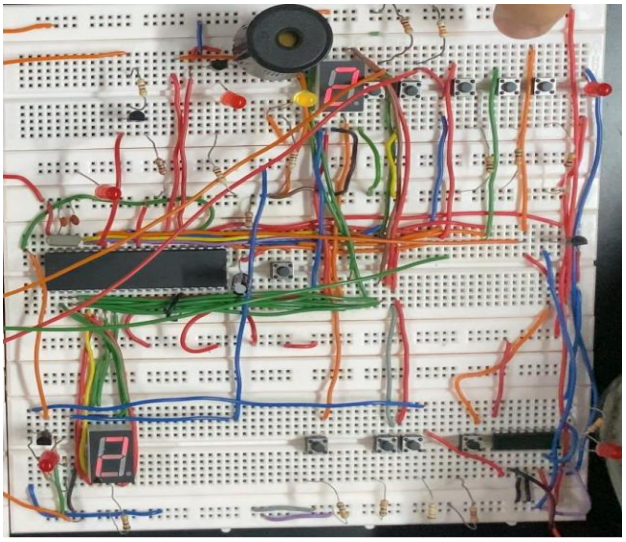


Figure 1.3: Seven Segment shown “2” when elevator system currently at floor 2

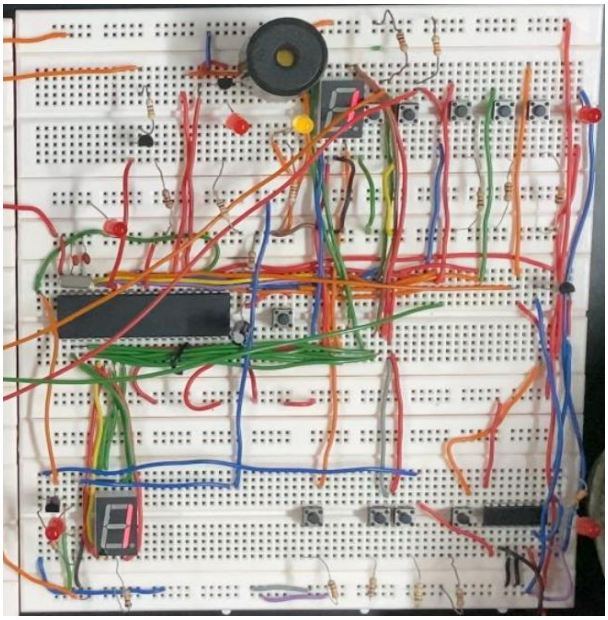


Figure 1.4: Seven Segment shown “1” when elevator system currently at floor 1

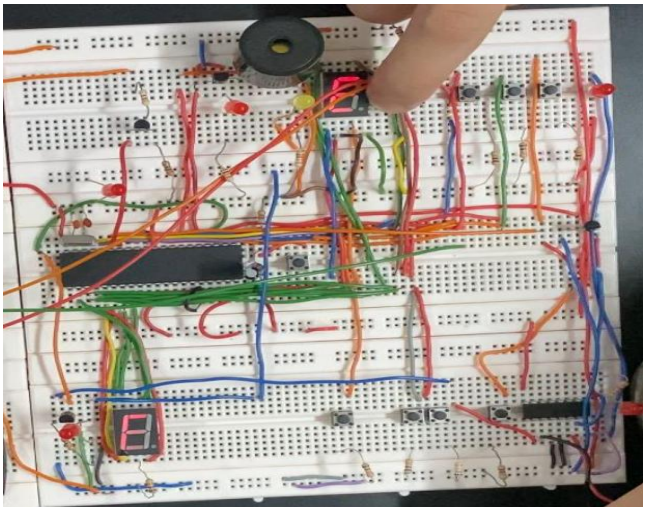


Figure 1.5: Seven Segment shown “E” when elevator system currently at Emergency mode

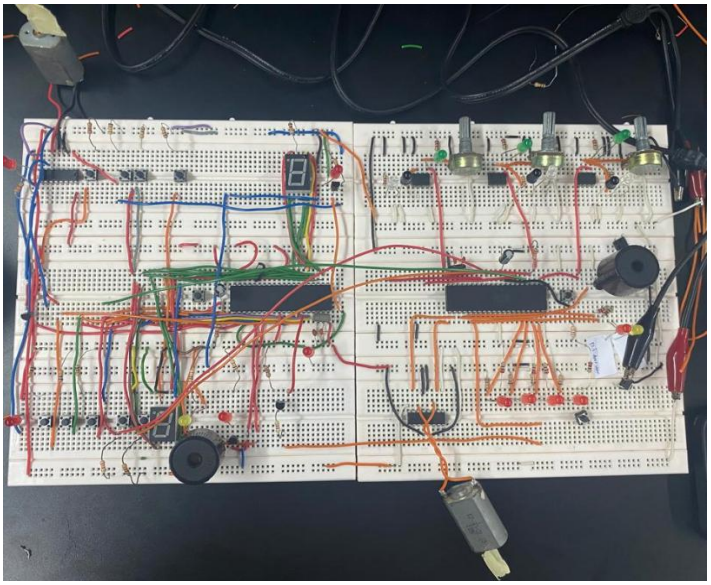


Figure 1.6: Final circuit after combining with topic 2, elevator door close/open system.

Assembly Program

```
ORG 000H ;Normal Operation starts here
LJMP SETUP
ORG 0003H; if resume button is pressed, it will return to here
LJMP RESUME
ORG 0013H; if emergency button is pressed it will start here
LJMP EMERGENCY
ORG 0030H; basic setup
SETUP:    MOV P1,#000H; P1 as output
          MOV R2,#003H; set current floor value to third floor, the current floor value is stored in R2
          MOV P3,#0FFH;set P3 as output
          MOV IE,#10000111B; enable the interrupt
          MOV IP,#00000001B; pioritize P3.2 over P3.3
          SETB IT0;proclaim that Interrupt 0(P3.2) is used
          SETB IT1;proclaim that Interrupt 1(P3.3) is used
          MOV P2,#09EH; P2 is incharged of displaying the seven segment display
          MOV R0,#09EH; R0 is incharged of storing the value of seven segment display
LOOP:     SETB P1.6;LIGHT UP idle LED
          MOV R5,#020H; first delay
          INTERNALLOOP2: MOV R4,#0FFH;second delay
          INTERNALLOOP3: MOV R3,#0FFH; third delay
          INTERNALLOOP:
          JNB P3.6, FLOOR2 ; floor 1, if P3.6 is enabled, jump to subroutine FLOOR 2
          JNB P3.5, FLOOR1 ; floor 2  if P3.5 is enabled, jump to subroutine FLOOR 1
          JNB P3.7, FLOOR3 ; floor 3  if P3.7 is enabled, jump to subroutine FLOOR 3
          DJNZ R3,INTERNALLOOP
          JNB P3.5, FLOOR1 ; floor 1
          JNB P3.6, FLOOR2 ; floor 2
          JNB P3.7, FLOOR3 ; floor 3
          DJNZ R4,INTERNALLOOP3
          JNB P3.7, FLOOR3 ; floor 1
          JNB P3.5, FLOOR1 ; floor 2
          JNB P3.6, FLOOR2 ; floor 3
          DJNZ R5,INTERNALLOOP2
          ACALL SLEEP ; if nothing has been triggred for a certain period of time, enter idle state
FLOOR1: ACALL OPEN1; enter open1 subroutine when Floor 1 subroutine is called
          MOV A, R2; move value of R2(current floor value) into A
          MOV B, #001H; Move 1 into B to prepare for subtraction
          SUBB A,B; subtract current floor by 1
          MOV R7,A; store the difference between floor into R7
          JZ EXIT1; if subtracted value is 0 jump to exit1 subroutine
          SETB P1.2; make the motor go up
          ACALL REPEAT; repeat for a certain times depending on the difference stored in R7
          CLR P1.2; Stop
          MOV R2,#001H; store current floor level(1) into R2
```

```
        MOV P2,#0CH; display 1 using seven segemnt
        MOV R0,#0CH; store "1" in R0
        ACALL OPEN1; open door again
        LJMP LOOP; go back to loop
        EXIT1: LJMP LOOP; go back to loop
        OPEN1: ACALL OPEN; go to open subroutine
                RET
FLOOR2: ACALL OPEN2; enter open2 subroutine when Floor 2 subroutine is called
        MOV A, R2; move value of R2(current floor value) into A
        MOV B, #002H; Move 2 into B to prepare for subtraction
        SUBB A,B ;subtract current floor by 2
        MOV R7,A; store the difference between floor into R7
        JC GOUP; if the value is negative go up
        JZ EXIT2; if subtracted value is 0 jump to exit2 subroutine
        SJMP GODOWN; if value is positive go down
        GOUP: MOV R7,#001H; set the repeat cycle number to 1
                SETB P1.3; turn on go up pin
                ACALL REPEAT; repeat the go up time by certain times depending on R7
                CLR P1.3; stop
                MOV R2,#002H; store current floor value(2) into R2
                MOV P2,#0B6H; display floor 2
                MOV R0,#0B6H; store value of display into R7
                ACALL OPEN2; enter open 2 subroutine
                LJMP LOOP; jump to loop
        GODOWN:MOV R7,#001H; set the repeat cycle number to 1
                SETB P1.2; turn on go down pin
                ACALL REPEAT; repeat the go down time by certain times depending on R7
                CLR P1.2 ;stop
                MOV R2,#002H; store current floor value(2) into R2
                MOV P2,#0B6H; display floor 2
                MOV R0,#0B6H; store value of display into R7
                ACALL OPEN2; enter open 2 subroutine
                LJMP LOOP; jump to loop
        EXIT2: LJMP LOOP; go to loop
        OPEN2: ACALL OPEN ; send a signal to part 2 to open the door
        RET
FLOOR3:  ACALL OPEN3 ;enter open3 sub routine when Floor3 subroutine is called
        MOV B, R2 ;move value of R2 (current floor value) into B
        MOV A, #003H; Move 3 into A to prepare for subtraction
        SUBB A,B ;subtract t=current floor by 3
        MOV R7,A
        JZ EXIT3; if subtracted value is 0 goto Exit3
        SETB P1.3 ;turn on go up pin
        ACALL REPEAT; repeatr for a certain cycle
        CLR P1.3; stop the go up pin
        MOV R2,#003H; store current floor value to R2
        MOV P2,#09EH; display 3 on seven segement
        MOV R0,#09EH; store display 3 on R0
        ACALL OPEN3; go to open 3 subroutine
        LJMP LOOP; goto loop
        OPEN3: ACALL OPEN ; open the door
        RET
        EXIT3: LJMP LOOP; go to loop
DELAY: MOV R5,#005H;delay subroutine
        SUBDELAY: MOV R4,#0FFH
        DELAYAGAIN: MOV R3,#0FFH
        DELAYMORE:
        DJNZ R3,DELAYMORE
        DJNZ R4,DELAYAGAIN
        DJNZ R5,SUBDELAY
        RET
OPEN:  SETB P1.1 ; send a signal to Part II to indicate they can start their part (open door subroutine)
        OPENLOOP: JNB P3.4, EXIT ; wait for another signal to be sent back to our Microcontroller
                SJMP OPENLOOP ; if no signal has been sent, stay idle
                EXIT: CLR P1.1 ; clear the signal sent to Part II
                RET
REPEAT: ACALL DELAY
        DJNZ R7, REPEAT; repeat the loop based on how the differenceb between floors
        RET
EMERGENCY:  MOV P2,#0F2H ;Display E on the seven segment display in emergency subroutine
        CLR P1.6 ;turn on internal LED
        CLR P1.3; stop the motor completely
        CLR P1.2
        SETB P1.4 ;BUZZER ON
        SETB P1.5 ;EMERGENCY LED
```

```
        MOV R5,#002H ;start a delay
        ESUBDELAY: MOV R4,#0FFH
        EDELAYAGAIN: MOV R3,#0FFH
        EDELAYMORE:DJNZ R3,EDELAYMORE
                        DJNZ R4,EDELAYAGAIN
                        DJNZ R5,ESUBDELAY
        CLR P1.4  ; stop the buzzer and LED
        CLR P1.5
        MOV R5,#002H        ;start a delay
        E2SUBDELAY: MOV R4,#0FFH
        E2DELAYAGAIN: MOV R3,#0FFH
        E2DELAYMORE:DJNZ R3,E2DELAYMORE
                        DJNZ R4,E2DELAYAGAIN
                        DJNZ R5,E2SUBDELAY
        JNB P3.2, RESET; continue the loop until resume button has been pressed.
        SJMP EMERGENCY
        RESET:; resume normal function
        RETI
RESUME: CLR P1.4 ;BUZZER OFF
        CLR P1.5 ;EMERGENCY LED OFF
        MOV P2,R0 ; Display current floor value
        MOV R7,#000H
        RETI
SLEEP: CLR P1.6 ;turn off internal LED
        JNB P3.5, SLOOP ;External floor 1
        JNB P3.6, SLOOP ;External floor 2
        JNB P3.7, SLOOP ;External floor 3
        SJMP SLEEP
        SLOOP: SETB P1.6 ; turn on internal LED
                LJMP LOOP ; go to loop
END
```

References

Agarwal, T. (2021) *8051 Microcontroller Architecture : Features & its applications*. https://www.elprocus.com/8051-microcontroller-architecture-and-applications/.

What is NPN Transistor? Definition, types & applications. (2021b). https://www.rs-online.com/designspark/what-is-npn-transistor-definition-types-applications#:~:text=Switching%20applications%20are%20where%20NPN,a%20current%20sink%20is%20required.

David (2023). *HUM: THE SIMPLE H-BRIDGE* - Soldered Electronics. [online] Soldered Electronics. Available at: https://soldered.com/learn/hum-the-simple-h-bridge/?srsltid=AfmBOooynEtqMAWqBIt-8bFAq2QdpEIGKcxAlXaOSwN7IjMMYJ0c2Jlp [Accessed 15 Sep. 2024].

Assignment Demo Link

https://drive.google.com/file/d/1AGf0ygK8S33uoC4mx_iAEpO0_52EePrK/view?usp=sharing